

Uso de `pidfd` para supervisão estável sem race de PID reciclado

Como construir a feature em um sistema Docker-like from scratch com Python, Cython e C++

Documento orientado a arquitetura, modelagem de domínio e implementação. O foco é um projeto em que o control plane, o catálogo de metadados e a API residem em Python, enquanto o data plane, a supervisão de processos e as interações de baixo nível com o kernel Linux ficam em C++, com Cython atuando como camada de ligação criteriosa entre os dois mundos.

Escopo da feature: supervisão estável baseada em `pidfd`, incluindo `pidfd_open`, `pidfd_send_signal` e espera via `waitid(P_PIDFD)` e `poll/epoll`.

Data da pesquisa: 12 de março de 2026.

Premissa operacional: hosts Linux modernos; baseline recomendada em produção: kernel 5.10 ou superior; baseline ideal para evolução: 6.9 ou superior.

Resumo executivo

O problema que o `pidfd` resolve não é cosmético. Em um runtime de containers, usar apenas um inteiro PID como identidade operacional do processo introduz um risco clássico de *time-of-check to time-of-use*: o runtime registra um PID, o processo original morre, o kernel recicla aquele número e uma operação tardia de `kill()`, `waitpid()` ou inspeção atinge o alvo errado. Em um sistema containerizado isso pode significar matar o processo errado no host, reportar o status incorreto do container, vaziar zombies, quebrar o contrato de restart policy ou corromper a semântica do `exec`. O `pidfd` converte a identidade do processo de um inteiro reciclável para um descritor de arquivo que o kernel trata como referência estável ao objeto processo. Essa é a mudança conceitual central desta feature [\[K1\]\[K2\]\[K3\]](#).

O segundo ponto decisivo é que `pidfd` não elimina a necessidade de arquitetura. O kernel permite observar a saída de um processo por `poll` ou `epoll` em um `pidfd`, mas a chamada `waitid(P_PIDFD)` só funciona quando aquele descritor se refere a um filho do processo chamador. Em outras palavras, a escolha de quem é o pai real do processo do container define se o runtime terá direito de esperar, recolher o status final e reaper o zombie de forma determinística. Essa restrição empurra o desenho para um supervisor de longa vida, preferencialmente um shim dedicado, e desaconselha que um processo Python de API ou um worker efêmero seja o dono direto do processo do container [\[K1\]\[K3\]\[K5\]](#).

No ecossistema Docker original, a direção arquitetural que se consolidou foi precisamente essa: o daemon do Docker passou a delegar o ciclo de vida ao `containerd`, que por sua vez conversa com um shim por task e com um runtime OCI como `runc`. O shim bloqueia a chamada de espera até a task sair, atua como subreaper e isola o ciclo de vida do processo do container das reinicializações do daemon. O `runc` moderno já começou a adotar `pidfd` em partes específicas, inclusive para expor o descritor do processo do container via `pidfd-socket`, mas a migração para espera totalmente orientada a `pidfd` ainda é incremental, o que abre espaço para um projeto novo nascer mais coerente do que a pilha histórica que precisou evoluir sem quebrar compatibilidade [\[D1\]\[D2\]\[CT1\]\[R1\]\[R2\]\[R3\]\[R4\]\[R5\]](#).

A recomendação central deste documento é a seguinte. Mantenha o control plane, a API, a orquestração de metadados, a autorização e a reconciliação em Python. Mantenha o data plane, a criação de processos, o manuseio de namespaces, cgroups, sinais, `pidfd`, timers e `epoll` em C++. Use Cython como cola fina e disciplinada, não como depósito de lógica de negócio. Mais especificamente, use Cython para empacotar um cliente local de alto desempenho e probes de capacidades do host, mas coloque a supervisão de longa duração em um processo C++ dedicado. Em termos práticos, a arquitetura recomendada é Python control plane + biblioteca C++ compartilhada + shim C++ por container/task + protocolo local sobre Unix domain socket. Esse desenho preserva o papel do Cython sem sacrificar a correção semântica exigida por `waitid(P_PIDFD)`, subreaping e crash recovery.

Mapa do documento

1. Escopo, contexto e tese arquitetural
 2. O problema real que pidfd resolve
 3. Conceitos fundamentais
 4. Como isso aparece no Docker original
 5. Implicações para o projeto Python, Cython e C++
 6. System Design proposto
 7. Low Level Design
 8. Domain Driven Design aplicado à feature
 9. Padrões de Design GoF que fazem sentido aqui
 10. Requirements funcionais sugeridos
 11. Requirements não funcionais sugeridos
 12. Plano de implementação em fases
 13. Riscos, edge cases e estratégia de testes
 14. Conclusão
- Apêndice A. Estrutura de módulos sugerida
- Apêndice B. Pseudocódigo do supervisor

1. Escopo, contexto e tese arquitetural

Esta pesquisa trata de uma feature específica, mas estrutural: como introduzir `pidfd` como mecanismo primário de supervisão de processos em um sistema Docker-like construído do zero. O objetivo não é apenas descrever as syscalls, e sim mostrar como elas alteram a forma correta de desenhar o runtime. A feature foi solicitada explicitamente no contexto de uma plataforma em que Python assume control plane, metadados e API, enquanto C++ concentra os comandos e as abstrações que dialogam com o kernel e o hardware do host. O tema, portanto, não é somente Linux internals; ele envolve desenho de fronteiras de responsabilidade, critérios de persistência, semântica operacional e governança da complexidade.

A tese do documento é que `pidfd` deve ser tratado como uma capacidade operacional efêmera do kernel, não como identidade de domínio. O domínio continua a girar em torno de entidades como *container*, *task*, *exec process*, *bundle*, *cgroup* e *runtime endpoint*. Já o `pidfd` é o handle correto para enviar sinais, observar a morte do processo, extrair descritores por `pidfd_getfd` em cenários avançados e acoplar o evento de saída ao event loop do supervisor. Essa distinção entre identidade de domínio e capacidade de infraestrutura evita um erro recorrente em projetos low level: modelar um detalhe do kernel como se fosse o identificador estável do negócio. Não é. Um `pidfd` não sobrevive ao reinício do processo supervisor, não atravessa persistência em disco e não deve aparecer como chave primária em repositórios de domínio.

A tese complementar é que a arquitetura precisa nascer já pensando em falhas parciais. Um projeto desse tipo costuma tropeçar menos no caminho feliz do que nas transições de falha: daemon Python reiniciado no meio de um stop; processo do container já morto quando a operação chega; namespace PID privado versus compartilhado; processo filho já zumbi; supervisor que precisa persistir o exit status antes de fazer o reap; exec concorrente; shutdown do host; kernels heterogêneos. A feature só estará realmente implementada quando o runtime for capaz de responder com determinismo em todos esses cenários, mantendo coerência entre estado observado no kernel, estado persistido no metadata store e estado exposto na API.

2. O problema real que `pidfd` resolve

O Linux tradicionalmente identifica processos por PIDs, que são números inteiros recicláveis dentro de um namespace. Em rotinas triviais isso parece suficiente, mas um runtime de containers opera em horizontes temporais maiores e com latência concorrente: cria processos, observa seu estado, reage a timers, reexecuta comandos, recebe chamadas remotas e convive com outras threads ou outros processos no host. Nesse contexto, um inteiro PID deixa de ser uma identidade segura e passa a ser uma referência instável. O problema clássico é conhecido há décadas: o runtime memoriza o

PID 1234, o processo 1234 termina, o kernel entrega o mesmo número a outro processo, e uma operação posterior que acreditava falar com o container original acaba interagindo com um processo completamente diferente.

Essa race é mais séria no domínio de containers do que em aplicações comuns porque o runtime possui autoridade para enviar sinais fortes, inclusive `SIGKILL`, criar e destruir cgroups, reportar exit codes para orquestradores e decidir se um workload deve ser reiniciado. O impacto não é apenas um bug local; ele pode quebrar contratos observáveis externamente. É por isso que o `runc` já contém comentários e proteções específicas para “avoid a PID reuse attack”, mesmo ainda usando caminhos baseados em PID em parte do código atual [\[R2\]](#)[\[R3\]](#).

O `pidfd` muda a mecânica porque transforma o processo em algo endereçável por descritor de arquivo. Como qualquer outro descritor, ele pode entrar em um `epoll`, obedecer à disciplina de ownership de FDs, ser transmitido explicitamente por `SCM_RIGHTS` se for necessário e falhar de maneira objetiva quando o objeto apontado deixa de existir. O resultado prático é que a sinalização passa a ser semanticamente estável: se o processo alvo já não existe, a operação falha com `ESRCH`; ela não “acerta outro processo” por acidente [\[K2\]](#).

Essa é a razão para não tratar `pidfd` como um simples detalhe de implementação. Ao introduzi-lo, o sistema deixa de ser orientado por “números de processo” e passa a ser orientado por handles do kernel. Isso aproxima a supervisão de processos da mesma filosofia usada para sockets, pipes, `eventfd`, `timerfd` e muitos outros recursos do Linux: entidades observáveis e controláveis por descritores estáveis. Em um runtime moderno, essa convergência não é uma conveniência, mas uma base arquitetural.

3. Conceitos fundamentais

3.1 `pidfd` como *capability* efêmera de processo

A documentação do kernel define `pidfd_open()` como a syscall que cria um descritor de arquivo que se refere à task associada a um PID. O descritor é devolvido já com `close-on-exec` habilitado, o que é importante tanto para higiene de descritores quanto para evitar vazamentos acidentais para o processo do container ou para subprocessos do próprio runtime [\[K1\]](#).

É útil insistir na palavra *capability*. O `pidfd` é uma capacidade operacional concedida pelo kernel ao processo supervisor. Ele não é a identidade de domínio do processo supervisionado; ele é a alavanca com que o supervisor age sobre o processo. Isso tem duas consequências de projeto. Primeiro, o `pidfd` deve ser encapsulado em infraestrutura e não “vazar” para a camada de domínio como identificador persistido. Segundo, o ciclo de vida do `pidfd` precisa ser modelado explicitamente como um recurso nativo: ownership único em C++, fechamento determinístico por RAII e ausência de cópias implícitas.

3.2 `pidfd_open`

`pidfd_open` existe desde Linux 5.3. A man page atual continua registrando que o glibc não fornece wrapper para a syscall, o que na prática significa que código C ou C++ precisa chamá-la via `syscall(SYS_pidfd_open, ...)` ou depender de uma biblioteca de terceiros que faça isso. Em Python a situação é diferente: a própria biblioteca padrão expõe `os.pidfd_open()`, o que mostra que o ecossistema Python reconheceu cedo o valor operacional do recurso ^{[K1][P1]}.

A nuance mais importante de `pidfd_open` é que, quando usado para um filho criado pelo próprio processo chamador, ainda existe uma janela conceitual entre o nascimento do filho e a abertura do descritor, embora a documentação esclareça que mesmo se o filho já tiver terminado seu PID não terá sido reciclado se certas condições forem verdadeiras: a disposição de `SIGCHLD` não pode ter sido alterada para `SIG_IGN`, `SA_NOCLDWAIT` não pode estar em vigor, e o zombie não pode ter sido reaped em outro local do programa ^[K1].

Essa ressalva é suficiente para duas decisões de arquitetura. Se o projeto busca o máximo de correção desde a primeira versão de produção, o caminho ideal é obter o `pidfd` atomicamente na criação do processo por meio de `clone3(CLONE_PIDFD)`. Se o projeto precisa de uma primeira iteração mais simples, `fork + pidfd_open` ainda pode ser aceitável dentro de um shim extremamente disciplinado que seja o único reaper, que não altere a semântica de `SIGCHLD` e que não tenha outras threads esperando filhos.

3.3 `CLONE_PIDFD` e `clone3`

`CLONE_PIDFD` foi introduzido em Linux 5.2. Quando usado com `clone3`, o kernel escreve o descritor do processo filho na estrutura de argumentos da própria syscall, removendo a etapa “crie primeiro, abra depois” e tornando atômico o vínculo entre criação do filho e obtenção do handle de supervisão ^[K4].

Para um runtime escrito em C++, essa é a interface mais alinhada ao objetivo da feature. Ela casa bem com um *launcher* nativo que já precisa manipular namespaces, cgroups, mounts, credenciais e `execve`. Também casa melhor com a separação proposta entre control plane em Python e data plane em C++, porque a parte C++ pode oferecer duas estratégias por trás da mesma interface: uma estratégia preferencial baseada em `clone3 + CLONE_PIDFD` e uma estratégia de compatibilidade baseada em `fork + pidfd_open`. Esse é um bom exemplo de onde um padrão Strategy agrega valor real e não apenas ornamentação.

3.4 `pidfd_send_signal`

`pidfd_send_signal`, disponível desde Linux 5.1, é a peça que substitui semanticamente o uso tradicional de `kill(pid, sig)`. A própria documentação do kernel é explícita ao afirmar que a syscall existe para evitar as races inerentes à sinalização por PID. Se o processo associado ao `pidfd` já terminou e foi esperado, a chamada falha com `ESRCH`, em vez de arriscar atingir um processo diferente que por acaso recebeu o mesmo número de PID ^[K2].

Há também uma nuance importante de namespaces. O chamador deve estar no mesmo namespace PID do alvo ou em um namespace ancestral. Para um runtime de containers isso reforça uma conclusão arquitetural simples: o supervisor deve permanecer no lado host da relação, ou em um namespace do qual o processo do container seja descendente visível. O shim não deve se aprisionar no namespace PID do container se quiser manter autoridade operacional limpa sobre o processo do workload ^{[K2][K6]}.

Desde Linux 6.9, a interface também ganhou flags para direcionar o escopo do sinal a thread específica, thread group ou process group. Para o caso mais comum de containers, em que a entidade supervisionada é o processo inicial do workload, normalmente basta o comportamento padrão, sobretudo se o `pidfd` se refere ao líder do grupo. Ainda assim, vale registrar que esses novos flags abrem espaço para ferramentas de exec, depuração e observabilidade mais finas no futuro ^[K2].

3.5 Espera por `pidfd`: `poll/epoll` e `waitid(P_PIDFD)`

Um dos aspectos mais elegantes do modelo é que o `pidfd` pode entrar em `poll`, `select` e `epoll`. Quando a task morre e se torna zombie, o descritor se torna legível; quando o processo é reaped, a interface de polling produz `EPOLLHUP`. Isso permite incorporar supervisão de processos ao mesmo reactor que observa sockets, pipes, `eventfd` e `timerfd`, sem depender de sinais assíncronos espalhados pelo código ^[K1].

Mas o detalhe mais importante está na man page de `wait(2)`: `waitid` aceita `P_PIDFD` desde Linux 5.4, porém essa espera só é válida quando o descritor se refere a um filho do processo chamador. A própria documentação de `pidfd_open` reforça isso. Em outras palavras, qualquer processo pode abrir um `pidfd` para observar a morte de outro processo, mas apenas o pai legítimo pode fazer o `wait` e recolher formalmente o status de saída daquele filho ^{[K1][K3]}.

Essa distinção precisa aparecer no desenho do software, não apenas na documentação. Para o supervisor, a sequência recomendada quando recebe `EPOLLIN` é fazer primeiro `waitid(P_PIDFD, ..., WEXITED | WNOWAIT)` para capturar um snapshot consistente do encerramento sem consumir imediatamente o zombie, persistir localmente essa informação de saída, publicar ou disponibilizar um evento para o control plane, e

somente então chamar novamente `waitid(P_PIDFD, ..., WEXITED)` para efetuar o reap definitivo. Esse uso em duas fases reduz a chance de o sistema perder a causa de saída em cenários de crash ou indisponibilidade parcial.

Quando o supervisor quiser integrar a espera a um modelo não bloqueante mais explícito, pode abrir o descritor com `PIDFD_NONBLOCK`. Nesse caso, uma chamada de `waitid` antes do término do processo retorna `EAGAIN`, o que é útil para evitar bloqueios acidentais em caminhos de inspeção ou em esperas condicionais. O Python expõe essa flag como `os.PIDFD_NONBLOCK` em versões recentes da biblioteca padrão ^{[K1][P1]}.

O Python expõe tanto `os.P_PIDFD` quanto `os.waitid`, e desde Python 3.12 o `asyncio` em Linux usa `PidfdChildWatcher` por padrão quando o host suporta `os.pidfd_open()`. Isso confirma que o recurso é maduro e útil também do lado Python. Ainda assim, isso não invalida a recomendação arquitetural deste documento: o fato de Python conseguir usar `pidfd` não significa que o processo Python de API deva ser o pai direto do workload ^{[P1][P3][P4]}.

3.6 Subreapers, orfandade e namespaces PID

A feature de `pidfd` conversa diretamente com outro conceito essencial em runtimes de containers: o subreaper. Um processo marcado com `PR_SET_CHILD_SUBREAPER` passa a cumprir, para seus descendentes órfãos, um papel semelhante ao do `init`. Processos cujo pai imediato morreu são reparentados ao ancestral subreaper vivo mais próximo, que então recebe `SIGCHLD` e pode chamá-los em `wait()` ^[K5].

No modelo de Docker e containerd, isso explica por que o shim é tão importante. O shim não é apenas um “proxy” do runtime; ele é o guardião da árvore de processos do container quando o daemon superior não deve ou não pode ser o pai direto. Em cenários de `setns`, `exec` e processos órfãos, essa responsabilidade de reaping é crucial. O README de runtime v2 do containerd deixa explícito que o shim `containerd-shim-runc-v2` atua como subreaper para limpar containers e processos relacionados ^[CT1].

Há ainda a semântica especial do processo `PID 1` dentro de um namespace PID privado. A documentação do kernel registra que o “init” do namespace só aceita sinais para os quais instalou handler, exceto `SIGKILL` e `SIGSTOP` enviados de um namespace ancestral; além disso, quando o processo `init` do namespace termina, os demais processos do namespace são mortos. Essa semântica ajuda a explicar por que matar o processo inicial do container costuma ser suficiente para derrubar o workload em um namespace PID privado, mas não resolve todos os cenários em namespaces compartilhados ^[K6].

3.7 Relação com cgroups e kill all

`pidfd` resolve identidade e supervisão do processo, mas não substitui o papel dos `cgroups` como mecanismo de escopo operacional. Em `cgroup v2` existe o arquivo `cgroup.kill`, que mata todos os processos do `cgroup` e dos descendentes por `SIGKILL`. Em um runtime realista, isso deve ser visto como complemento e não concorrente do `pidfd`. O `pidfd` controla de forma precisa o processo-alvo; o `cgroup` controla o conjunto de processos pertencentes ao container ^[K7].

A regra prática recomendada é simples. Para *stop gracioso*, o caminho principal é `pidfd_send_signal(SIGTERM)` no processo inicial. Para *force kill*, o runtime precisa considerar o isolamento usado pelo container. Se ele tiver namespace PID privado e a semântica do produto aceitar derrubar o `init` como mecanismo suficiente, isso costuma bastar. Se houver namespace PID compartilhado, processos auxiliares externos ao `init` ou requisito de “kill all” estrito, o runtime deve acionar `cgroup.kill` ou um fallback equivalente. O próprio `runc` atual mantém lógica de matar todos os processos do `cgroup` em alguns cenários, o que confirma que `pidfd` não elimina a necessidade de raciocínio em nível de grupo ^{[R2][K7]}.

4. Como isso aparece no Docker original

4.1 Daemons, runtimes e a separação em camadas

O Docker histórico não nasceu com a topologia que hoje parece “óbvia”. A documentação de releases registra que, a partir do Docker 1.11, a instalação Linux deixou de ser um binário monolítico e passou a ser composta por quatro binários: `docker`, `docker-containerd`, `docker-containerd-shim` e `docker-runc`. Esse detalhe histórico é relevante porque ele mostra o momento em que o ecossistema aceitou, na prática, que um runtime de containers robusto precisa de separação entre daemon de controle, supervisor de processo e executor OCI ^[D1].

A documentação atual do `dockerd` reforça o mesmo desenho sob outra forma. O daemon do Docker depende de um runtime compatível com OCI, invocado via `containerd`, como interface para namespaces, `cgroups` e SELinux, usando `runc` por padrão. O próprio Docker também documenta que shims do `containerd` instalados no `PATH` podem ser usados diretamente. Em outras palavras, a arquitetura oficial já assume um plano de controle separado do mecanismo que efetivamente ancora o processo do container no host ^[D2].

4.2 O papel do shim e a semântica de espera

O README de runtime v2 do `containerd` mostra o fluxo de alto nível com clareza: o `containerd` cria a task no shim, inicia a task e depois emite um `WaitRequest` que bloqueia até a task sair. O documento também recomenda o uso de `ttrpc` em shims para reduzir memória e tamanho binário, uma decisão coerente com a natureza minimalista e per-task desses processos [\[CT1\]](#).

Mais importante do que o protocolo em si é a semântica: quem “espera” de verdade é o shim. O daemon superior quer o resultado da espera, não a responsabilidade processual de fazer o wait. Essa separação é exatamente o que um projeto novo deveria internalizar ao desenhar uma feature baseada em `pidfd`. O supervisor que recebe o evento do kernel e faz o reap precisa ser a peça mais próxima do filho, não necessariamente a peça mais rica em regras de negócio.

O mesmo README ainda registra explicitamente o papel de subreaper do `containerd-shim-runc-v2`. Se o container compartilha o namespace PID com o shim, descendentes órfãos serão reparentados ao shim e ele fará o reap. O texto também chama atenção para o detalhe de que não há reparenting cruzando namespaces em certos cenários de `setns`, o que mostra o quanto a supervisão correta depende de entender as regras do kernel e não apenas de “manter uma lista de PIDs” [\[CT1\]](#)[\[K5\]](#)[\[K6\]](#).

4.3 Onde o runc já usa pidfd e onde ainda não é end-to-end

O `runc` contemporâneo já contém sinais claros de adoção de `pidfd`. Há código em `libcontainer/init_linux.go` que abre um `pidfd` do processo `init` com `unix.PidfdOpen(os.Getpid(), 0)` e envia o descritor de volta por socket. Também há suporte de linha de comando para obter o `pidfd` do container via a flag `pidfd-socket`, registrada no changelog recente [\[R1\]](#)[\[R2\]](#)[\[R4\]](#).

Ao mesmo tempo, partes importantes do código ainda operam por PID e por objetos de processo da biblioteca padrão de Go. Há caminho em que a sinalização delega a `p.cmd.Process.Signal(sig)`, e o próprio código contém comentários defensivos para evitar ataques de reuso de PID antes de sinalizar um container “não running”. Esse contraste é revelador: o `runc` está numa transição. Ele reconhece o valor do `pidfd`, mas precisa coexistir com caminhos legados e restrições de compatibilidade [\[R2\]](#)[\[R3\]](#).

Há ainda um pull request aberto com a proposta explícita de “try to use pidfd and epoll to wait init process exit”. O simples fato de essa melhoria ainda estar em evolução em 2025 e permanecer visível em março de 2026 mostra que migrar um runtime estabelecido para uma espera totalmente orientada a `pidfd` não é trivial. Isso fortalece a recomendação para o seu projeto: em vez de repetir o legado histórico e depois remendar, projete desde o início uma supervisão em que `pidfd`, `epoll` e a topologia do shim sejam conceitos de primeira classe [\[R5\]](#).

5. Implicações para o projeto Python, Cython e C++

5.1 Por que o processo Python de API não deveria ser o pai real do workload

À primeira vista, a disponibilidade de `os.pidfd_open()`, `os.P_PIDFD` e `signal.pidfd_send_signal()` em Python poderia sugerir que o próprio serviço Python de API poderia criar e supervisionar diretamente os processos dos containers. Tecnicamente isso é possível em um protótipo. Arquiteturalmente, porém, é uma escolha fraca para produção. O motivo principal é a restrição semântica de `waitid(P_PIDFD)`: quem quiser esperar e reaper o filho precisa ser o pai dele. Um processo de API costuma ter ciclos de vida mais voláteis, múltiplos workers, reinicializações por deploy e preocupações de isolamento de falha incompatíveis com a função de “guardião da árvore de processos” [\[K1\]\[K3\]\[P1\]\[P2\]](#).

Há um segundo motivo. Quando o processo que é pai real do workload morre, a árvore é reparentada ao subreaper apropriado ou ao init. Nesse ponto, um novo processo Python até pode reabrir um `pidfd` com `pidfd_open` para observar o processo sobrevivente, mas já não tem direito de fazer o *wait* daquele filho se ele não for mais seu descendente direto. O sistema perde a posse da semântica completa de saída. Isso significa que um design em que o processo Python é o supervisor principal dificulta justamente o que runtimes reais mais precisam: sobreviver a reinícios do control plane sem perder a governança sobre o ciclo de vida da task.

Há ainda um terceiro motivo, mais sutil e igualmente importante: mistura de concerns. O processo que fala HTTP ou gRPC, valida payloads, consulta metadados, autentica chamadas, registra auditoria e executa lógica de reconciliação não deveria também acumular a responsabilidade de ficar em um reactor de FDs, tratando `EPOLLIN` de `pidfd`, timers de grace period e pipes de stdio. Isso degrada separação de camadas, amplia o raio de falha e torna mais difícil raciocinar sobre backpressure, shutdown gracioso e observabilidade.

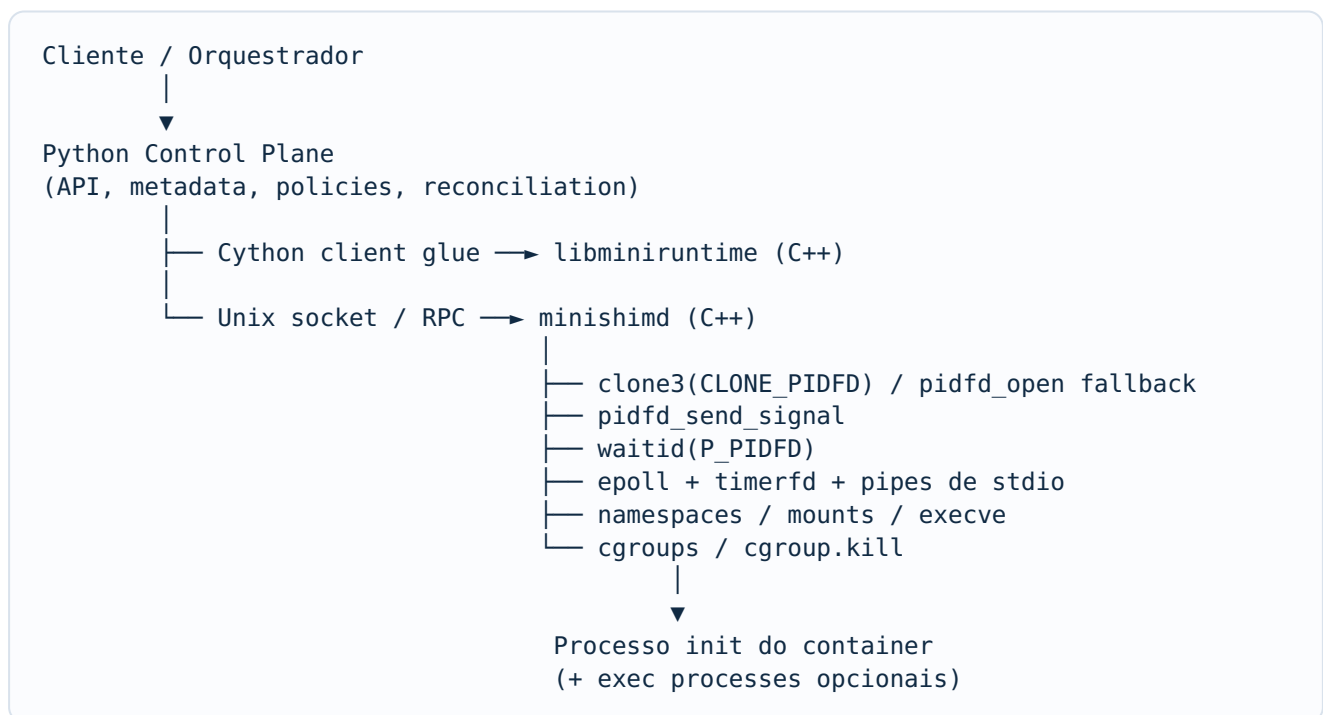
5.2 A arquitetura recomendada

A arquitetura recomendada para o seu projeto é composta por quatro peças centrais. A primeira é o *control plane* em Python, responsável por API externa, orquestração de lifecycle, metadados, autorização, políticas, reconciliação e exposição de observabilidade. A segunda é uma biblioteca C++ compartilhada, que concentra as abstrações de kernel, wrappers de syscall, modelagem de recursos nativos, serialização de requests low level e código reutilizável pelo shim. A terceira é um shim C++ de longa vida, idealmente um processo por container ou por task, que se torna o pai real

do workload e executa a supervisão operacional. A quarta é uma camada Cython fina, usada para ligar Python à biblioteca C++ em operações locais como feature probing, marshaling eficiente, composição de requests e chamada de um cliente nativo do shim.

Esse desenho preserva a ideia original do projeto. Python continua com a inteligência do sistema. C++ continua com o contato de baixo nível com o kernel. Cython continua sendo a cola entre os dois. A diferença é que a cola não é usada de maneira ingênua para manter o workload como filho direto do processo Python. Em vez disso, ela ajuda Python a coordenar um componente C++ que assume a responsabilidade processual que o kernel exige para uma supervisão correta.

Em termos operacionais, o fluxo sugerido é o seguinte. O serviço Python recebe uma solicitação de *create/start*, valida a intenção, persiste uma intenção inicial no metadata store e aciona uma operação local via Cython para compor o request de runtime e lançar um shim C++ dedicado. O shim cria namespaces, prepara cgroups, abre os FDs necessários, lança o processo inicial do container com `clone3(CLONE_PIDFD)` quando possível, registra o `pidfd` em seu reactor `epoll`, escreve seu estado local em um diretório de runtime e expõe um socket local para que o control plane Python consulte estado, peça sinais, faça wait lógico e acompanhe a vida da task. Se o control plane reiniciar, o shim permanece vivo e a relação pai-filho do kernel não é perdida.



Do ponto de vista de evolução, esse arranjo também facilita a interoperabilidade. O controle em Python fica livre para falar com vários shims, hospedar múltiplos endpoints de API, suportar políticas de restart e eventualmente lidar com runtimes alternativos. O data plane em C++ fica pequeno, testável e próximo do modelo do kernel. E o Cython permanece focado em FFI e desempenho, sem carregar acidentalmente a semântica vital de ownership de processo.

6. System Design proposto

6.1 Topologia de processos e responsabilidades

A topologia recomendada não deve nascer de gosto pessoal, e sim de propriedades do kernel. O processo Python principal não precisa e não deve ancorar diretamente os processos do container. Em vez disso, ele deve gerenciar *shims* C++ que sejam estáveis o suficiente para sobreviver a reinícios do control plane. Cada shim pode supervisionar um único container ou um conjunto muito pequeno de tasks correlatas. Para um projeto nascente, um shim por container é a escolha mais simples de raciocinar, mais fácil de isolar e mais próxima do padrão que já se provou robusto no ecossistema. Um shim compartilhado por muitos containers economiza processos, mas amplia o raio de blast em caso de falha e complica muito a modelagem de recursos e shutdown.

O control plane em Python deve manter um registro de *runtime endpoints* ativos, cada um apontando para o socket Unix do shim, o PID do shim, o diretório de estado local e um identificador estável do container. O store persistente do control plane não deve guardar o inteiro bruto do `pidfd` do processo init, porque esse FD é local ao processo shim e não faz sentido fora dele. O que o metadata store deve guardar é um conjunto de fatos reconciliáveis: `container_id`, `shim_pid`, `shim_socket_path`, `init_host_pid`, `init_start_time`, `cgroup_path`, timestamps relevantes e o último estado conhecido.

Essa distinção entre estado persistente e handle efêmero é um princípio de design central. Um runtime robusto nunca confunde “o que precisa sobreviver ao reinício do processo” com “o que existe apenas enquanto um processo está vivo”. O `pidfd` pertence à segunda categoria. O fato de o process init ter host PID X, start time Y e estar associado ao container Z pertence à primeira.

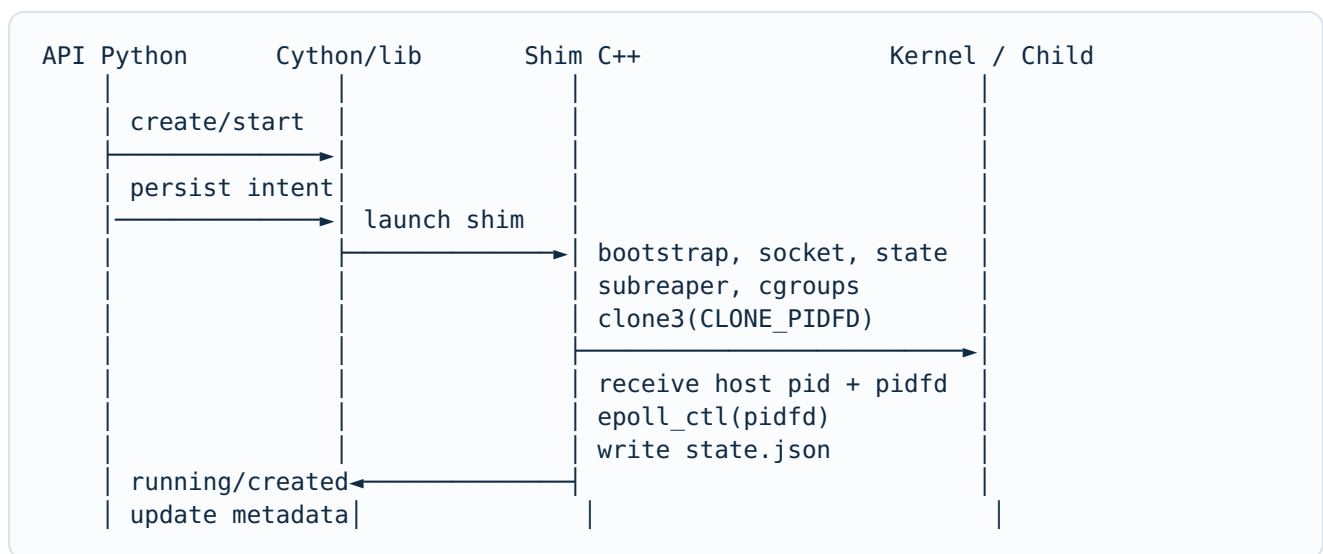
6.2 Fluxo de *create/start*

O fluxo de criação começa no control plane Python. A API recebe o pedido, valida o bundle OCI, calcula políticas de recursos, resolve a imagem ou rootfs, reserva um diretório de estado e persiste um registro inicial do container em estado “creating”. Em seguida, por meio da camada Cython e da biblioteca C++, o control plane lança o shim e entrega a ele a configuração já normalizada. O shim então executa a sequência de baixo nível: cria seu socket de administração, bloqueia os sinais que serão tratados por FDs, marca-se como subreaper, prepara cgroups, monta ou ajusta namespaces necessários, organiza stdio e cria o processo init do container.

O ponto ideal para capturar o `pidfd` é a própria criação do filho com `clone3(CLONE_PIDFD)`. Se essa estratégia não estiver disponível naquele host, o shim usa o caminho de compatibilidade por `fork + pidfd_open`, respeitando a disciplina de não haver outro reaper concorrente. Assim que o processo init existe e o `pidfd` foi adquirido, o shim registra o FD em seu `epoll`, coleta o host PID, obtém o `start_time` de `/proc/<pid>/stat` para defesa adicional contra ambiguidades de reuso e grava um

snapshot inicial de estado em disco. Só então responde ao control plane Python que o container passou a “created” ou “running”, conforme a semântica escolhida para o projeto.

Essa ordem é importante. Não é seguro anunciar que o container está operacional antes de o supervisor ter de fato seu handle operacional do kernel registrado no reactor. Caso contrário, um cliente externo poderia chamar *stop* ou *inspect* em uma janela em que o container já existe, mas o runtime ainda não tem sua referência estável. Em termos de consistência, o ponto de transição para “running” deve ser posterior ao momento em que o `pidfd` entrou no conjunto de FDs supervisionados.



6.3 Fluxo de *stop*, *kill* e *wait*

O caminho de parada deve ser desenhado como um protocolo explícito. Quando o control plane decide parar um container, ele envia ao shim um comando que contém o sinal inicial, o grace period e a política de escopo. O shim localiza a entidade *ProcessInstance* do `init`, verifica seu estado interno e chama `pidfd_send_signal` no `pidfd` correspondente. Se o alvo já tiver saído e sido esperado, a syscall retornará `ESRCH`; nesse caso o comando deve ser tratado como idempotente ou como “already finished”, e não como erro catastrófico [K2].

Se o sinal inicial foi gracioso, normalmente `SIGTERM`, o shim arma um `timerfd` com o grace period. Caso o `pidfd` se torne legível antes do timer expirar, o shim entra no caminho de coleta de saída. Se o timer vence sem saída, o shim eleva a força para `SIGKILL`. Em seguida, dependendo da política do container, ele pode acionar também `cgroup.kill` para garantir encerramento de todo o grupo de processos. Para containers em namespace PID privado, matar o `init` normalmente já destrói o namespace; para namespaces compartilhados ou contratos de “kill all”, o `cgroup` é o escopo certo para fechamento definitivo [K6][K7].

O caminho de espera lógica também deve ser pensado. O control plane não precisa necessariamente ficar bloqueado no kernel; ele pode delegar ao shim um *Wait RPC* de alto nível, inspirado no modelo do containerd, que bloqueia até a saída ser conhecida e então retorna status, sinal e timestamps. Isso preserva a semântica de pai-filho no shim e oferece ao control plane uma API limpa. O README de runtime v2 do containerd usa precisamente essa ideia ao descrever um `WaitRequest` que bloqueia no shim até a task sair ^[CT1].

6.4 Persistência local, crash recovery e reinício do control plane

Um desenho realmente sólido assume que o control plane Python cairá em algum momento enquanto o shim e o workload continuam vivos. Por isso o shim precisa ter um diretório de estado próprio, preferencialmente em `/run` para estado efêmero e possivelmente em `/var/lib` para artefatos que devam sobreviver a reboot conforme o produto exigir. O mínimo recomendado é um `state.json` com informações correntes do container e um `exit.json` escrito atômica e idempotentemente quando a saída é observada.

No momento em que o `pidfd` indica término, o shim deve usar `waitid(..., WNOWAIT)` para capturar o snapshot de saída, persistir `exit.json` por escrita atômica seguida de `rename()`, e só depois chamar o wait final que consome o zombie. Essa ordem é essencial porque o control plane pode estar fora do ar quando o processo sai. O shim não pode depender de um roundtrip com Python para tornar o reap seguro. A persistência local é a âncora de consistência entre o mundo do kernel e o eventual retorno do plano de controle.

Quando o processo Python reinicia, ele executa uma rotina de reconciliação. Essa rotina vasculha o conjunto de shims conhecidos, tenta reconectar a cada socket vivo e reconstrói o estado de cada container a partir da combinação entre metadata store e estado local do shim. Para tarefas ainda em execução, o control plane pode ler do shim o host PID e, se necessário, revalidar o processo via `start_time` e cgroup path. Para tarefas já encerradas, o `exit.json` é suficiente para remontar a visão de domínio. O detalhe crítico é que a autoridade operacional do reap nunca saiu do shim, portanto a reinicialização do Python não degrada a semântica do runtime.

O cenário de queda do próprio shim é diferente e precisa ser explicitado. Se o shim, que é o pai do processo init, morrer antes do filho, a árvore de processos será reparentada segundo as regras do kernel, e um processo substituto poderá no máximo reabrir um `pidfd` para observação, mas não herdará automaticamente o direito de fazer wait daquele filho. Isso significa que o crash do shim é uma falha mais grave do que o crash do control plane. Em consequência, o projeto deve tratar o shim como componente pequeno, altamente testado e supervisionado pelo init do host ou por um service manager robusto.

6.5 Suporte a `exec`, múltiplos processos e namespaces compartilhados

Um erro comum é desenhar a feature como se cada container tivesse apenas um processo relevante: o `init`. Na prática, um sistema Docker-like também precisa lidar com `exec`. Cada processo criado por `exec` deve ser tratado como uma entidade operacional separada, com seu próprio host PID, seu próprio `pidfd` quando possível, seu próprio conjunto de pipes de `stdio` e seu próprio ciclo de vida de saída. O aggregate de *Container* precisa, portanto, possuir ao menos um *InitProcess* e zero ou muitos *ExecProcess*.

Isso importa especialmente em namespaces PID compartilhados. Matar o `init` do container pode não ser suficiente para garantir a saída de todos os processos associados ao escopo lógico do container. É por isso que o design proposto recomenda que a política de stop seja declarativa e explicita se o escopo desejado é “apenas `init`”, “todos os processos do `cgroup`” ou “todos os processos conhecidos do aggregate”. O kernel fornece os blocos básicos; o produto precisa escolher a semântica visível ao usuário.

6.6 Feature detection e compatibilidade de kernel

O suporte a `pidfd` é suficientemente granular para exigir detecção explícita. Linux 5.1 introduz `pidfd_send_signal`; Linux 5.2 adiciona `CLONE_PIDFD`; Linux 5.3 traz `pidfd_open`; Linux 5.4 permite `waitid(P_PIDFD)`; Linux 5.10 introduz `PIDFD_NONBLOCK`; Linux 6.9 amplia o escopo de sinalização com novos flags [\[K1\]](#)[\[K2\]](#)[\[K3\]](#)[\[K4\]](#)[\[P1\]](#).

Em um produto real, não basta inspecionar `uname -r`. O runtime deve fazer *probing* ativo, porque um kernel novo ainda pode estar sob políticas de `seccomp` ou de `sandbox` que bloqueiem certas `syscalls`. A biblioteca C++ deve montar um objeto *KernelFeatureSet* durante a inicialização do host agent ou do shim, tentando chamadas reais em ambiente controlado e registrando quais caminhos são suportados. Esse objeto pode então subir ao control plane Python para influenciar decisões de agendamento, warnings de compatibilidade e escolha de estratégia de launcher.

6.7 Alinhamento com a máquina de estados OCI

Embora o seu projeto seja “Docker-like” e não necessariamente uma implementação literal do stack Docker, vale a pena manter alinhamento conceitual com o modelo OCI. A runtime spec define estados como *creating*, *created*, *running* e *stopped*, e a configuração também prevê fases de hooks como `createRuntime`, `createContainer`, `startContainer`, `poststart` e `poststop` [\[O1\]](#)[\[O2\]](#).

Isso sugere que o seu desenho interno pode ser um pouco mais rico do que a máquina de estados pública. Internamente, é perfeitamente útil distinguir “`pidfd` vinculado mas processo ainda não promovido a *running*”, “saída observada mas ainda não *reaped*” e

“stopped projetado para o metadata store”. Externamente, porém, a API pode continuar expondo uma visão compatível com OCI. Essa abordagem permite rigor operacional sem apresentar ao usuário final toda a granularidade de infraestrutura.

7. Low Level Design

7.1 Componentes C++ recomendados

O desenho de baixo nível deve ser orientado a recursos nativos e ownership explícito. O primeiro bloco que vale a pena criar é um tipo *UniqueFd*, pequeno e trivial, com semântica de movimento, fechamento automático no destrutor, impossibilidade de cópia e utilitários para `dup`, `release` e inspeção segura. Quase todo o resto do runtime depende disso. Sem uma abstração desse tipo, o código tende a acumular vazamentos, confusão de ownership e bugs de fechamento duplo.

Sobre esse bloco base, o runtime deve ter wrappers ou facades mínimas para `Pidfd`, `EpollReactor`, `TimerFd`, `UnixServerSocket`, `CgroupManager`, `NamespaceLauncher` e `StateDirWriter`. Em vez de espalhar chamadas cruas a `syscall()`, `openat()`, `write()` e `ioctl()` por todo o código, é preferível concentrá-las em um pequeno conjunto de adapters com semântica clara. Isso melhora testabilidade e prepara o terreno para mocks ou fakes em testes unitários.

Outra decisão importante é o modelo de erro. Em C++ moderno, o runtime se beneficia de um tipo estilo `std::expected` ou equivalente, capaz de transportar tanto o valor de retorno quanto um erro rico que preserve `errno`, contexto textual e, se necessário, campos estruturados. Exceções podem ser aceitáveis em algumas camadas, mas na fronteira com syscalls e no protocolo com o shim tende a ser melhor produzir erros explícitos e serializáveis.

7.2 Event loop do shim

O núcleo do shim deve ser um reactor baseado em `epoll`. Um event loop single-threaded é, aqui, uma força e não uma limitação. Como a carga é I/O-bound e os eventos relevantes já são orientados a descritores de arquivo, um único thread de reactor simplifica o raciocínio sobre ordering, elimina locks desnecessários ao redor de estado de processo e torna mais previsível a relação entre eventos do kernel e transições internas. A partir desse loop, o shim observa ao menos quatro tipos de FD: o socket de administração, os `pidfds` dos processos supervisionados, os `timerfds` de grace period e, opcionalmente, `signalfd` para sinais que não sejam convenientemente modelados apenas por `pidfd`.

Essa escolha aproxima o projeto do estilo de runtimes maduros e conversa bem com a própria natureza do `pidfd`. Se o processo morre, um evento chega ao `epoll`. Se um cliente pede stop, uma mensagem chega ao socket. Se o grace period vence, um evento

chega ao `timerfd`. O loop central não precisa de polling ativo, `sleep` arbitrário nem handlers de sinal dispersos. A supervisão fica naturalmente declarativa: cada FD representa uma fonte legítima de estado do kernel.

7.3 Algoritmo de start

O algoritmo de criação do processo `init` pode ser expresso em poucos passos conceituais. O shim constrói um objeto imutável de configuração de launch, prepara `cgroups` e descritores, cria a task com `clone3(CLONE_PIDFD)` quando possível, registra o `pidfd` no reactor, coleta metadados de verificação como `start_time`, escreve o snapshot inicial de estado e só então expõe sucesso ao control plane. Caso o `execve` no filho falhe, o erro precisa atravessar de volta ao pai de forma inequívoca, por exemplo por um pipe de erro dedicado fechado com `CLOEXEC` no caminho feliz.

O detalhe que merece destaque é que o objeto *ProcessInstance* deve nascer já com a associação entre `container_id`, host PID, `start_time`, `pidfd`, escopo de `cgroup` e enum de estado interno. Não crie estruturas paralelas em mapas independentes para cada um desses dados. Em LLD, fragmentar estado do processo em várias coleções sem `aggregate` explícito é receita para inconsistência.

7.4 Algoritmo de stop e coleta de saída

O algoritmo de parada deve ser literal e previsível. Recebido um comando de stop, o shim valida se a instância ainda está em estado compatível com sinalização. Se estiver, chama `pidfd_send_signal` com o sinal configurado. Em caso de sucesso, arma o timer de `grace` se aplicável. Quando o `pidfd` sinaliza saída, o shim chama `waitid(P_PIDFD, ..., WEXITED | WNOWAIT)`, converte o `siginfo_t` em um objeto *ExitStatus*, persiste localmente o resultado, marca o estado como “exit observed” e então faz o wait final para reaping. Esse desenho separa o instante em que o kernel informa a morte do processo do instante em que o runtime considera o evento totalmente comprometido em sua própria durabilidade.

Se o timer expira antes da saída, o shim emite `SIGKILL` via `pidfd_send_signal` e, se a política exigir, aciona `cgroup.kill`. A combinação entre `pidfd` e `cgroup` oferece o melhor dos dois mundos: precisão para o processo inicial e cobertura de conjunto para o escopo lógico do container.

```
EPOLLIN(pidfd)
├─ waitid(P_PIDFD, fd, &info, WEXITED | WNOWAIT)
│   ├── traduz info para ExitStatus
│   ├── grava exit.json por rename atômico
│   ├── publica evento local / responde Wait RPC
│   └─ waitid(P_PIDFD, fd, &info, WEXITED)
│       └─ reaping concluído
```

7.5 Fallbacks e estratégias por versão de kernel

O runtime deve abstrair a estratégia de launcher e a estratégia de kill. Na criação, a estratégia preferencial é `clone3 + CLONE_PIDFD`; a de compatibilidade é `fork + pidfd_open`. Na parada forçada, a estratégia preferencial em cgroup v2 é `cgroup.kill`; a de compatibilidade pode ser congelar, enumerar PIDs do cgroup e sinalizar cada processo ou depender da semântica do namespace PID privado. Ao encapsular essas variantes atrás de interfaces pequenas, o projeto reduz o número de condicionais espalhadas pelo código e torna a compatibilidade um problema local, não sistêmico.

7.6 Papel do Cython

A camada Cython deve ser deliberadamente fina. A documentação de Cython mostra o caminho usual para envolver C++ com `cdef extern from`, `cdef cppclass` e métodos expostos em módulos `.pyx`. Também mostra como declarar exceções C++ traduzíveis para Python e como liberar o GIL em trechos de código nativo que façam I/O bloqueante ou computação significativa [\[C1\]\[C2\]\[C3\]](#).

No seu projeto, isso sugere uma divisão muito concreta. Cython deve expor ao Python um cliente nativo do shim, tipos de request/response de baixo overhead e um pequeno módulo de detecção de recursos do host. Chamadas potencialmente bloqueantes ao shim devem ocorrer em seções `with nogil` ou em funções nativas que não dependam do runtime Python. Por outro lado, a lógica de domínio, as transições de estado do container, as políticas de restart, a persistência em banco e a reconciliação não devem migrar para Cython. Esse código ganha pouco com FFI e perde muito em clareza, testabilidade e velocidade de iteração.

É igualmente importante que a camada Cython não receba ownership de FDs do shim sem necessidade. O controle do `pidfd` deve continuar no processo C++ que é pai do workload. Se o Python quiser inspecionar ou solicitar ações, ele deve fazê-lo por protocolo, e não tentando tornar-se dono indireto de recursos cuja semântica depende do processo supervisor correto.

```
# cy_runtime_client.pyx (esboço conceitual)

cdef extern from "runtime_client.hpp" namespace "mini":
    cdef cppclass RuntimeClient:
        RuntimeClient(const string& socket_path) except +
        StartReply start(const StartRequest&) except +
        StopReply stop(const string& container_id, int timeout_ms) except +
        InspectReply inspect(const string& container_id) except +

    cdef class PyRuntimeClient:
        cdef RuntimeClient* _cli

        def stop(self, container_id: str, timeout_ms: int):
            cdef StopReply reply
            with nogil:
```

```
reply = self._cli.stop(container_id.encode(), timeout_ms)
return decode_stop_reply(reply)
```

8. Domain Driven Design aplicado à feature

8.1 Bounded contexts

O erro mais comum ao aplicar DDD em software de runtime é confundir “baixo nível” com “sem domínio”. Mesmo um sistema profundamente acoplado ao kernel possui linguagem ubíqua, invariantes de negócio e fronteiras de contexto. No desenho proposto, há ao menos quatro bounded contexts naturais. O primeiro é *Lifecycle Orchestration*, em Python, onde vivem comandos como create, start, stop, kill, exec, delete e restart, assim como as políticas visíveis ao usuário. O segundo é *Process Supervision*, em C++, onde vivem os detalhes de pidfd, espera, reaping, timers e escopo de sinais. O terceiro é *Host Capability Discovery*, que modela o conjunto de features do host e informa decisões arquiteturais. O quarto é *Metadata Projection*, onde a visão operacional de baixo nível é projetada em uma visão persistente, consultável e eventualmente auditável para a API.

Separar esses contextos não é academicismo. Quando o control plane pergunta “o container está running?”, ele não está pedindo um dump bruto de `siginfo_t`; ele quer uma projeção de domínio. Quando o shim decide se deve chamar `waitid` com `WNOWAIT`, ele não está decidindo política de restart; está cumprindo uma regra de infraestrutura. Misturar essas linguagens cria modelos confusos e APIs erráticas.

8.2 Aggregate root e entidades

O aggregate central deve ser o *Container*. Ele é a raiz que controla o ciclo de vida observável, o vínculo com o bundle OCI, a associação com cgroup e rootfs, e a coleção dos processos pertencentes ao seu escopo lógico. Dentro desse aggregate, o *InitProcess* e os *ExecProcesses* são entidades, não value objects, porque possuem identidade operacional ao longo do tempo e transições próprias de estado.

Já tipos como *HostPid*, *StartTimeTicks*, *ExitStatus*, *SignalPolicy*, *GracePeriod*, *CgroupPath*, *BundlePath* e *RuntimeEndpoint* são excelentes candidatos a value objects. O `pidfd` em si, entretanto, não deve aparecer como entity nem como value object de domínio persistente. Ele deve ficar encapsulado em um adaptador de infraestrutura, algo como *PidfdHandle*, de visibilidade restrita ao bounded context de supervisão.

Essa distinção dá origem a uma regra importante de modelagem: nunca derive a identidade da entidade *ProcessInstance* do próprio `pidfd`. A identidade deve ser algo como “init” ou um `exec_id` estável; o `pidfd` é apenas o recurso pelo qual o supervisor conversa com a entidade no kernel.

8.3 Domain services

Há alguns serviços de domínio que merecem existir acima das entidades. Um *LifecycleService* em Python decide se uma transição é permitida e qual comando operacional deve ser emitido. Um *ReconciliationService* reconstitui a visão de domínio a partir de metadata store, estado local do shim e inspeção do host. Um *KernelFeaturePolicy* avalia se um host atende aos requisitos necessários para determinadas garantias. No lado C++, um *SupervisorService* orquestra o reactor e aplica a semântica correta de reaping.

Perceba que nem tudo precisa ser “entidade”. A lógica que transforma um evento low level de saída em uma transição coesa do aggregate pertence mais naturalmente a serviços e fábricas de domínio do que ao objeto que embrulha o FD. Essa separação mantém o aggregate limpo e evita que objetos de infraestrutura comecem a ditar linguagem de domínio.

8.4 Domain events

Esta feature se beneficia muito de eventos explícitos. *ContainerCreated*, *InitProcessSpawned*, *InitPidfdBound*, *StopRequested*, *GraceTimeoutElapsed*, *ProcessExitObserved*, *ProcessReaped*, *ContainerStopped* e *ControlPlaneReconciled* são exemplos naturais. O objetivo não é “event sourcing total”, mas clareza de transição. Quando o shim recebe `EPOLLIN` no `pidfd`, isso ainda não precisa significar que o aggregate já está em “stopped”. Pode significar apenas que a saída foi observada e está sendo comprometida. Esse refinamento é útil para não mentir sobre durabilidade.

Se o sistema precisar integrar-se com outros componentes, como um scheduler ou um coletor de auditoria, a melhor prática é usar uma projeção de eventos ou um *outbox* transacional no control plane, não acoplar diretamente a lógica do shim a filas remotas. O shim precisa ser pequeno e resiliente; ele não deve depender de brokers externos para poder concluir um reap.

9. Padrões de Design GoF que fazem sentido aqui

Em um sistema desse tipo, padrões GoF só são úteis quando reduzem acoplamento real. O primeiro padrão que vale a pena aplicar é *Facade*. O control plane Python deve enxergar uma interface compacta para o data plane, algo como `start()`, `stop()`, `inspect()`, `wait()`, `exec()` e `delete()`. A fachada esconde do restante do sistema os detalhes de sockets Unix, serialização, `pidfd`, timers e políticas de fallback.

O segundo padrão natural é *Strategy*. Ele se aplica muito bem à escolha do launcher e do mecanismo de kill. Um *SpawnStrategy* pode ter uma implementação *Clone3PidfdStrategy* e outra *ForkPidfdOpenStrategy*. Um *KillStrategy* pode ter

variantes para *init-only*, *cgroup-kill* e *hybrid kill*. Isso permite que o sistema se adapte a diferentes kernels sem contaminar o domínio com condicionais de compatibilidade espalhadas.

O terceiro padrão útil é *State*. O aggregate *Container* tem transições internas suficientemente relevantes para justificar um modelo explícito de estado, em vez de múltiplos booleanos combinados. “Creating”, “Created”, “Running”, “Stopping”, “ExitObserved”, “Stopped” e “Deleted” formam uma máquina de estados natural. Além de clareza, isso evita operações sem sentido, como emitir um novo stop em uma instância já reaped mas ainda não projetada como deleted.

O quarto padrão é *Command*. Solicitações como create, start, stop, kill e exec podem ser modeladas como comandos imutáveis que trafegam do control plane para o shim. Isso ajuda em auditoria, teste e idempotência, especialmente quando o sistema precisar reenfileirar operações após reconciliação.

O quinto padrão é *Observer*, embora na prática ele deva ser materializado como um pequeno barramento interno ou como callbacks de eventos. Quando o shim observa uma saída, múltiplos interessados podem reagir: projeção de metadados, streaming para API de eventos, métricas e políticas de restart. O importante é que o shim não codifique dependências rígidas para cada consumidor.

Também há espaço para *Adapter* e *Bridge*. *Adapter* é o padrão certo para encapsular syscalls e arquivos de cgroup. *Bridge* descreve bem a separação entre o modelo Python de lifecycle e a implementação C++ de supervisão operacional, conectados por Cython e por um protocolo local. Por fim, *Abstract Factory* pode ser aplicado na construção de um *LaunchContext* adequado ao tipo de workload, ao isolamento desejado e às capacidades do host.

O cuidado importante é não transformar o projeto num catálogo de padrões. O valor está em nomear poucos padrões certos, nas fronteiras certas, para manter o software compreensível enquanto cresce.

10. Requirements funcionais sugeridos

RF-01 — Criação com vínculo operacional estável. O sistema deve criar o processo init do container de forma que o supervisor obtenha um handle estável de processo antes de declarar o container operacional. O caminho preferencial é `clone3(CLONE_PIDFD)`; o fallback aceitável é `fork + pidfd_open` sob invariantes estritas de reaping ^{[K1][K4]}.

RF-02 — Sinalização sem race de PID reciclado. O sistema deve suportar envio de sinais ao init e aos processos de exec usando `pidfd_send_signal`, com semântica idempotente para casos em que o processo já tenha encerrado. A API deve refletir de forma explícita a diferença entre “container inexistente”, “processo já finalizado” e “sinal não permitido” ^[K2].

RF-03 — Espera e coleta de saída. O sistema deve ser capaz de bloquear logicamente até a saída do `init` ou de um processo de `exec`, retornando código de saída, sinal, timestamps e causa conhecida do término. Essa espera deve ser realizada pelo shim pai do processo, e não por workers arbitrários do control plane [K1][K3][CT1].

RF-04 — Persistência do motivo de saída. Antes do reap definitivo, o supervisor deve persistir localmente o snapshot de saída. O metadata store do control plane deve então projetar essa informação em sua visão persistente, preservando a distinção entre “saída observada” e “saída reconciliada”.

RF-05 — Stop gracioso com timeout. O sistema deve suportar *stop* com *grace period* configurável, emitindo sinal gracioso inicialmente e escalando para encerramento forçado quando o timer expirar. O comportamento deve ser determinístico e auditável.

RF-06 — Escopo configurável de kill. O sistema deve permitir que o produto escolha se um *force kill* se aplica apenas ao processo `init`, a todos os processos do cgroup ou a outro escopo explicitamente definido. Essa semântica deve ser visível na API e não ficar implícita em detalhes de namespace.

RF-07 — Suporte a `exec`. O sistema deve tratar processos de `exec` como entidades de lifecycle independentes, com criação, observação, `stdio` e saída próprios, sem reduzir toda a modelagem ao processo `init`.

RF-08 — Reconciliação após reinício do control plane. Quando o processo Python reiniciar, ele deve ser capaz de reconectar-se aos shims vivos, reconstruir estado corrente e continuar servindo inspeções, waits lógicos e comandos de lifecycle sem exigir recriação do container.

RF-09 — Descoberta de capacidades do host. O sistema deve expor, por API administrativa ou introspecção interna, quais hosts suportam cada parte da feature: `pidfd_open`, `waitid(P_PIDFD)`, `CLONE_PIDFD`, `PIDFD_NONBLOCK` e `cgroup.kill`.

RF-10 — Observabilidade operacional. O sistema deve gerar logs estruturados, eventos e métricas suficientes para responder perguntas como: quando o `stop` foi pedido, qual sinal foi enviado, quando o processo saiu, quando o reap ocorreu, se houve escalonamento para `SIGKILL` e qual caminho de compatibilidade foi usado naquele host.

11. Requirements não funcionais sugeridos

RNF-01 — Correção semântica acima de conveniência. Em hosts que suportem a feature, o runtime não deve regredir para sinalização baseada apenas em PID quando existir `pidfd` disponível. O uso do handle estável deve ser a rota normal, não um detalhe opcional.

RNF-02 — Isolamento de falha. Queda do control plane Python não pode implicar perda da relação pai-filho do kernel nem incapacidade de concluir a supervisão e o reap. Queda do shim deve ser tratada como falha grave, rara e observável, com o menor raio de blast possível; por isso a preferência por um shim pequeno e por container.

RNF-03 — Idempotência operacional. Comandos de stop, kill, wait e inspect devem ter semântica idempotente ou claramente classificável em presença de retries de rede, reinícios e eventos de saída já ocorridos.

RNF-04 — Higiene de descritores. FDs usados pelo runtime devem ser criados com `CLOEXEC` quando apropriado, encapsulados por ownership único e protegidos contra vazamento para o processo do container. O próprio `pidfd` já nasce com essa propriedade, mas a disciplina precisa abranger o restante dos descritores ^[K1].

RNF-05 — Compatibilidade progressiva. O sistema deve operar em kernels heterogêneos com degradação controlada e explicitamente registrada, em vez de assumir silenciosamente que todas as garantias estão presentes. Quando o host não suportar um requisito necessário para determinada garantia, isso deve estar visível para operadores e componentes superiores.

RNF-06 — Baixo acoplamento entre domínio e kernel. O modelo de domínio não deve depender de detalhes como números de FD, formato de `siginfo_t` ou paths internos de `/proc`. Essas dependências pertencem ao bounded context de supervisão e devem chegar à camada de domínio já traduzidas.

RNF-07 — Observabilidade de transições. O runtime deve permitir reconstruir, em logs e métricas, a linha do tempo exata entre criação do shim, aquisição do `pidfd`, entrada em running, pedido de stop, expiração de grace period, saída observada e reap. Sem isso, incidentes de lifecycle viram adivinhação.

RNF-08 — Testabilidade. O código deve ser estruturado para suportar testes unitários com fakes de syscall, testes de integração com kernels diversos e testes de estresse que forcem churn alto de processos para expor falhas de reuso de PID.

RNF-09 — Segurança operacional. Perfis de seccomp, capabilities e políticas do host precisam permitir as syscalls necessárias ao runtime, mas o componente deve operar com o menor conjunto de privilégios viável. Além disso, o socket de administração do shim deve estar protegido por permissões e ownership corretos, já que quem consegue falar com ele ganha poder operacional sobre containers.

RNF-10 — Eficiência. O supervisor não deve usar busy waiting nem loops baseados em `sleep`. O modelo correto é orientado a eventos de FD. Isso reduz overhead, simplifica escalabilidade e se alinha à natureza do recurso `pidfd`.

12. Plano de implementação em fases

Fase 0 — Fundação de infraestrutura. Comece criando a biblioteca C++ base com `UniqueFd`, wrappers mínimos de `syscall`, reactor `epoll`, serialização de mensagens e um pequeno conjunto de testes unitários para ownership de FDs e tradução de erros. Em paralelo, defina no Python o aggregate `Container`, o repositório de metadados e as interfaces de aplicação de lifecycle. Nessa fase, o objetivo não é ainda iniciar containers reais, mas estabilizar as fronteiras entre as camadas.

Fase 1 — Shim mínimo com start e inspect. Implemente um shim C++ capaz de receber um request de launch, iniciar um processo simples com namespace/cgroup controlados, obter `pidfd`, registrar o handle no `epoll` e responder a uma chamada de inspeção. Nessa etapa, já vale a pena materializar o diretório de estado local e a lógica de bootstrap do shim a partir do control plane Python.

Fase 2 — Stop, wait e persistência de saída. Introduza `pidfd_send_signal`, timers de grace period, `waitid(P_PIDFD)` em duas fases com `WNOWAIT` e persistência local de `exit.json`. Nesse momento o sistema já deve ser capaz de demonstrar a principal promessa da feature: sinalizar e esperar sem race de PID reciclado.

Fase 3 — Cgroups e semântica de kill all. Adicione suporte completo a `cgroup.kill` quando presente e fallbacks para cenários em que ele não exista. Formalize em API a diferença entre stop gracioso e force kill de escopo amplo. Essa fase fecha a lacuna entre supervisão do processo e supervisão do escopo lógico do container.

Fase 4 — Exec e múltiplos processos. Estenda o aggregate e o shim para suportar processos de `exec` com lifecycle individual. Essa fase costuma revelar rapidamente onde o modelo foi erroneamente simplificado ao redor do `init`, por isso é valiosa mesmo que o produto inicial não exponha `exec` ao usuário final.

Fase 5 — Reconciliação e tolerância a reinícios. Construa a rotina de reinício do control plane, reatachando metadados a shims vivos e reconstruindo estado a partir do diretório de runtime. Teste exaustivamente o cenário “o Python caiu, o container não”. Só depois disso a feature pode ser considerada pronta para ambientes reais.

Fase 6 — Hardening e otimização. Nessa fase entram melhorias como adoção mandatória de `clone3(CLONE_PIDFD)` onde disponível, observabilidade avançada, suporte a flags de sinalização mais recentes, revisão de `seccomp`, benchmarking e refino de footprint do shim. É também aqui que padrões de compatibilidade com kernels legados podem ser revisados com dados concretos de produção.

13. Riscos, edge cases e estratégia de testes

Há alguns riscos que merecem atenção explícita. O primeiro é tratar `pidfd` como se fosse “persistível”. Não é. Reiniciar o processo supervisor invalida o FD anterior, e um inteiro representando o número do descritor não tem significado fora do processo que o abriu. O segundo risco é acreditar que `pidfd_open` sozinho resolve todos os cenários de criação; para filhos recém-criados, a garantia mais forte vem de `CLONE_PIDFD`. O terceiro risco é subestimar a diferença entre observar a morte de um processo e ter autoridade para fazer o wait dele.

Outro edge case importante é o de políticas de `SIGCHLD`. A documentação de `pidfd_open` deixa claro que a garantia de não reciclagem do PID para um filho já terminado, no caminho `fork + pidfd_open`, depende de não usar `SIG_IGN` para `SIGCHLD`, de não habilitar `SA_NOCLDWAIT` e de não ter outro reaper concorrente. Isso deve virar regra codificada, não conhecimento tribal ^[K1].

Também é preciso testar cenários de namespace. Quando o init do namespace PID morre, os demais processos do namespace são mortos, mas isso não substitui a necessidade de políticas de escopo explícitas para namespaces compartilhados ou processos externos ao init. Cenários de `setns`, processos órfãos e interação com subreaper devem aparecer em testes de integração, porque são justamente os casos em que a semântica intuitiva costuma falhar ^{[K5][K6][CT1]}.

A estratégia de testes deve ter pelo menos quatro camadas. A primeira é unitária, com fakes e adapters de syscall para validar transições internas e ownership de recursos. A segunda é de integração em host Linux real, cobrindo create, stop, kill, wait, restart do control plane e múltiplos processos de exec. A terceira é de compatibilidade, executando a suíte em kernels representativos, idealmente 5.4, 5.10, 6.1 e 6.9+ em VMs ou CI especializada. A quarta é de estresse, com churn intenso de processos para forçar reuso de PID e demonstrar que o runtime não sinaliza o processo errado.

Há ainda um tipo de teste particularmente valioso para essa feature: o teste destrutivo de falha parcial. Mate o processo Python de API enquanto o container roda. Depois reinicie o control plane e confirme que o shim continua supervisionando, que o estado é reconciliado corretamente e que um stop posterior ainda funciona. Em outro experimento, force a queda do shim e observe o impacto real. Esses experimentos revelam mais sobre a qualidade do design do que dezenas de testes de caminho feliz.

14. Conclusão

Introduzir `pidfd` em um sistema Docker-like não é apenas trocar uma syscall por outra. É mudar a unidade de verdade operacional do runtime. O sistema deixa de confiar em números de PID recicláveis e passa a operar sobre handles do kernel adequados a

sinalização, observação e integração com reactors baseados em FDs. Essa mudança resolve uma classe real de bugs e vulnerabilidades de semântica, mas só entrega todo o valor quando a arquitetura respeita a restrição fundamental de `waitid(P_PIDFD)`: o supervisor que espera e reapela precisa ser o pai do processo supervisionado [K1][K2][K3].

No contexto do seu projeto, isso conduz a uma recomendação firme. Python deve continuar como o cérebro do sistema: API, metadados, orquestração, políticas e reconciliação. C++ deve continuar como o braço operacional do sistema: criação de processos, namespaces, cgroups, sinais, timers e reactor de baixo nível. Cython deve continuar como cola, mas como cola bem posicionada, ligando o control plane a uma biblioteca e a clientes nativos, não como substituto para um shim de longa vida. Em outras palavras, a feature “uso de pidfd” não cabe confortavelmente em um desenho puramente in-process entre Python e C++; ela pede uma topologia em que um shim C++ seja o guardião efetivo do processo do container.

O Docker, o containerd e o runc chegaram a essa vizinhança por evolução histórica. O seu projeto tem a vantagem de poder começar já de onde o ecossistema mais experiente aprendeu, às vezes de forma dolorosa, que precisava estar. Se a implementação seguir a separação proposta, modelar pidfd como capability efêmera, persistir somente fatos reconciliáveis, usar o shim como pai real e manter o domínio limpo em Python, a feature não será apenas correta: ela se tornará uma das bases estruturais sobre as quais o restante do runtime poderá crescer com muito menos dívida técnica.

Apêndice A. Estrutura de módulos sugerida

Uma distribuição pragmática de código pode seguir a estrutura abaixo. No lado Python, um pacote `control_plane` contém API, aplicação, domínio, repositórios e projeções. O módulo `runtime_client`, implementado em Cython, oferece ao Python uma fachada de alto desempenho para falar com o shim e para executar probes de capacidade. No lado C++, a biblioteca `libminiruntime` concentra wrappers de syscall, tipos RAIL, cgroup manager, launcher, parser de requests e serializer de replies. O executável `minishimd` liga essa biblioteca ao event loop, ao state dir e ao protocolo local. Essa separação permite reuso sem empurrar o coração do supervisor para dentro do processo Python.

```
python/  
  control_plane/  
    api/  
    application/  
    domain/  
    repository/  
    reconciliation/  
    projections/
```

```

runtime_client/
  cy_runtime_client.pyx
  cy_kernel_features.pyx

cpp/
  libminiruntime/
    fd.hpp
    pidfd.hpp
    syscalls.hpp
    epoll_reactor.hpp
    timerfd.hpp
    cgroup_manager.hpp
    launcher.hpp
    state_dir.hpp
    protocol/
  minishimd/
    main.cpp
    shim_server.cpp
    task_supervisor.cpp
    exec_supervisor.cpp

```

Apêndice B. Pseudocódigo do supervisor

```

// stop gracioso com escalonamento

Result Supervisor::stop(ContainerId id, int timeout_ms) {
    auto& proc = registry_.init_process(id);
    if (!proc.is_signalable()) {
        return Result::AlreadyStopped;
    }

    int rc = syscall(SYS_pidfd_send_signal, proc.pidfd.fd(), SIGTERM, nullptr, 0);
    if (rc == -1 && errno == ESRCH) {
        return Result::AlreadyStopped;
    }
    if (rc == -1) {
        return ErrnoError("pidfd_send_signal(SIGTERM)", errno);
    }

    reactor_.arm_timer(proc.grace_timer_fd.fd(), timeout_ms, [this, id] {
        auto& proc = registry_.init_process(id);
        syscall(SYS_pidfd_send_signal, proc.pidfd.fd(), SIGKILL, nullptr, 0);
        if (features_.has_cgroup_kill && proc.kill_scope == KillScope::Cgroup) {
            cgroups_.kill_tree(proc.cgroup_path);
        }
    });

    return Result::Accepted;
}

// observação de saída e reap em duas fases

void Supervisor::on_pidfd_readable(ProcessInstance& proc) {
    siginfo_t info{};
    if (waitid(P_PIDFD, proc.pidfd.fd(), &info, WEXITED | WNOWAIT) == -1) {

```

```

        log_errno("waitid WNOWAIT", errno);
        return;
    }

    ExitStatus status = translate_siginfo(info);
    state_dir_.write_exit_atomically(proc.container_id, status);
    protocol_.publish_exit(proc.container_id, status);

    if (waitid(P_PIDFD, proc.pidfd.fd(), &info, WEXITED) == -1) {
        log_errno("waitid reap", errno);
        return;
    }

    proc.mark_reaped(status);
}

```

Referências

As referências abaixo foram usadas para fundamentar definições, contextualização histórica, detalhes de kernel, comportamento de bibliotecas e decisões arquiteturais. Sempre que possível, foram priorizadas fontes primárias: manual pages do Linux, documentação oficial do Python, documentação oficial do Docker, especificações OCI, documentação do kernel e código ou documentação oficial de containerd e runc.

- [K1] **pidfd_open(2) — Linux manual page.** man7.org. URL: https://man7.org/linux/man-pages/man2/pidfd_open.2.html. Acesso em 12 mar. 2026.

- [K2] **pidfd_send_signal(2) — Linux manual page.** man7.org. URL: https://man7.org/linux/man-pages/man2/pidfd_send_signal.2.html. Acesso em 12 mar. 2026.

- [K3] **wait(2) — Linux manual page.** man7.org. URL: <https://man7.org/linux/man-pages/man2/wait.2.html>. Acesso em 12 mar. 2026.

- [K4] **clone(2) — Linux manual page.** man7.org. URL: <https://man7.org/linux/man-pages/man2/clone.2.html>. Acesso em 12 mar. 2026.

- [K5] **PR_SET_CHILD_SUBREAPER(2const) — Linux manual page.** man7.org. URL: https://man7.org/linux/man-pages/man2/pr_set_child_subreaper.2const.html. Acesso em 12 mar. 2026.

- [K6] **pid_namespaces(7) — Linux manual page.** man7.org. URL: https://man7.org/linux/man-pages/man7/pid_namespaces.7.html. Acesso em 12 mar. 2026.

[K7]

Control Group v2 — Linux Kernel documentation. docs.kernel.org. URL: <https://docs.kernel.org/admin-guide/cgroup-v2.html>. Acesso em 12 mar. 2026.

[P1] **os — Python 3 documentation.** docs.python.org. URL: <https://docs.python.org/3/library/os.html>. Acesso em 12 mar. 2026.

[P2] **signal — Python 3 documentation.** docs.python.org. URL: <https://docs.python.org/3/library/signal.html>. Acesso em 12 mar. 2026.

[P3] **What's New In Python 3.9.** docs.python.org. URL: <https://docs.python.org/3/whatsnew/3.9.html>. Acesso em 12 mar. 2026.

[P4] **What's New In Python 3.12.** docs.python.org. URL: <https://docs.python.org/3/whatsnew/3.12.html>. Acesso em 12 mar. 2026.

[C1] **Using C++ in Cython.** cython.readthedocs.io. URL: https://cython.readthedocs.io/en/latest/src/userguide/wrapping_CPlusPlus.html. Acesso em 12 mar. 2026.

[C2] **Interfacing with External C Code.** cython.readthedocs.io. URL: https://cython.readthedocs.io/en/latest/src/userguide/external_C_code.html. Acesso em 12 mar. 2026.

[C3] **Thread states and the global interpreter lock.** docs.python.org. URL: <https://docs.python.org/3/c-api/threads.html>. Acesso em 12 mar. 2026.

[D1] **Docker Engine prior releases.** docs.docker.com. URL: <https://docs.docker.com/engine/release-notes/prior-releases/>. Acesso em 12 mar. 2026.

[D2] **dockerd reference.** docs.docker.com. URL: <https://docs.docker.com/reference/cli/dockerd/>. Acesso em 12 mar. 2026.

[CT1] **containerd runtime v2 README.** GitHub / containerd. URL: <https://github.com/containerd/containerd/blob/main/core/runtime/v2/README.md>. Acesso em 12 mar. 2026.

[R1] **runc libcontainer/init_linux.go.** GitHub / opencontainers/runc. URL: https://github.com/opencontainers/runc/blob/main/libcontainer/init_linux.go. Acesso em 12 mar. 2026.

[R2] **runc libcontainer/container_linux.go.** GitHub / opencontainers/runc. URL: https://github.com/opencontainers/runc/blob/main/libcontainer/container_linux.go. Acesso em 12 mar. 2026.

- [R3] **runc libcontainer/process_linux.go**. GitHub / opencontainers/runc. URL: https://github.com/opencontainers/runc/blob/main/libcontainer/process_linux.go. Acesso em 12 mar. 2026.
-
- [R4] **runc CHANGELOG**. GitHub / opencontainers/runc. URL: <https://github.com/opencontainers/runc/blob/main/CHANGELOG.md>. Acesso em 12 mar. 2026.
-
- [R5] **runc pull requests**. GitHub / opencontainers/runc. URL: <https://github.com/opencontainers/runc/pulls>. Acesso em 12 mar. 2026.
-
- [01] **OCI Runtime Spec**. specs.opencontainers.org. URL: <https://specs.opencontainers.org/runtime-spec/runtime/>. Acesso em 12 mar. 2026.
-
- [02] **OCI Runtime Spec Config**. specs.opencontainers.org. URL: <https://specs.opencontainers.org/runtime-spec/config/>. Acesso em 12 mar. 2026.
-